

Technical Skills & Research Portfolio

Diego Octavio Rodríguez Ruiz

Nanotechnology Engineering Student | University of Guadalajara
Specializing in Computational Chemistry and Data Analysis

COMPUTATIONAL CHEMISTRY & MOLECULAR MODELING

Quantum Mechanical Simulations (ORCA)

Adsorption Study of Small Molecules on Olympipene Surfaces

- **Objective:** Evaluated physisorption properties of CH_4 , CO , and CO_2 on an olympipene model (proxy for functionalized graphene).
- **Methodology & Rigor:**
 - Implemented **D3BJ (Grimme's dispersion correction)** to accurately capture Van der Waals interactions.
 - Utilized **RIJCOSX** approximation to optimize SCF cycle efficiency without loss of accuracy.
 - Performed **Frequency Analysis (FREQ)** to validate optimized structures as true potential energy surface minima (confirmed by zero imaginary frequencies).

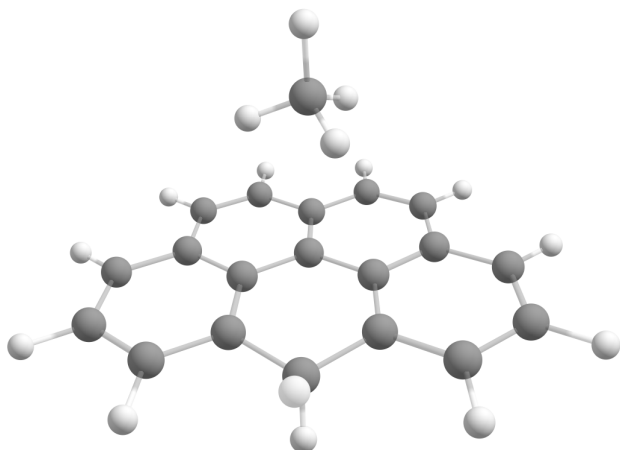


Figure 1: **Methane adsorption on Olympipene.** Optimized geometry showing the physisorption of CH_4 on the central ring of an olympipene model. The simulation was performed at the B3LYP/6-31G level of theory, incorporating Grimme's D3BJ dispersion correction to accurately model long-range Van der Waals interactions.

Listing 1: ORCA Input for Olympipene-Methane adsorption study (B3LYP/6-31G)

```
! RKS B3LYP D3BJ RIJCOSX 6-31G def2/J OPT FREQ

%pal
  nprocs 4
end

%maxcore 8000

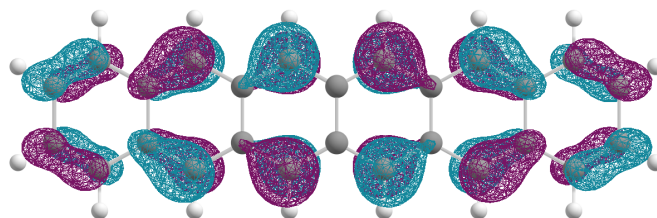
* xyz 0 1
[Coordinates omitted for brevity]
*
```

ELECTRONIC STRUCTURE ANALYSIS OF POLYCYCLIC AROMATIC HYDROCARBONS (PAHS)

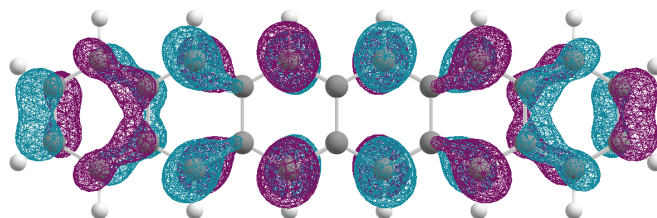
- **Systematic Study:** Analyzed the evolution of electronic properties in linear acenes, from Benzene ($n = 1$) to Hexacene ($n = 6$).
- **Computational Workflow:**
 - Multi-step simulation pipeline: Geometry optimization and frequency validation (OPT/FREQ), followed by Single Point energy and Molecular Orbital (MO) localization.
 - Implemented **Time-Dependent DFT (TD-DFT)** to calculate the first vertical excitation root, determining the optical band gap.
- **Key Findings:** Observed the characteristic red-shift in the absorption spectrum as a function of the number of rings, correlating with the narrowing of the HOMO-LUMO gap due to increased π -conjugation.

Table 1: Electronic properties and HOMO-LUMO Gap evolution for the studied acene series.

Rings	Molecule	HOMO (eV)	LUMO (eV)	Gap (eV)
1	Benzene	-6.68	-0.42	6.26
2	Naphthalene	-5.83	-1.13	4.71
3	Anthracene	-5.42	-1.55	3.87
4	Tetracene	-5.13	-1.85	3.28
5	Pentacene	-4.91	-2.07	2.84
6	Hexacene	-4.76	-2.23	2.53



LUMO (Orbital 90)



HOMO (Orbital 89)

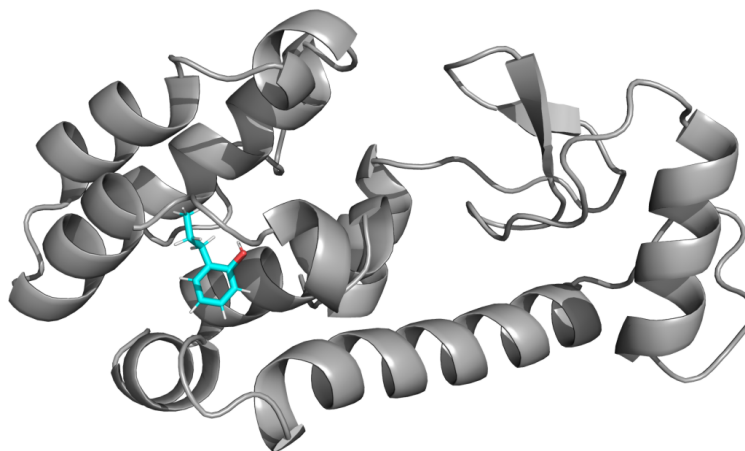
Figure 2: **Distribución espacial de los orbitales de frontera.** Comparativa vertical que resalta la simetría y los planos nodales del hexaceno.

Listing 2: TD-DFT configuration for excited state analysis (n

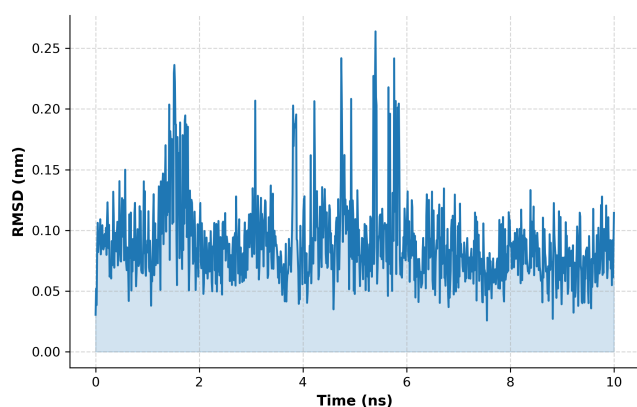
```
%tddft
nroots 1 # Calculating the first vertical excitation
end
! LibXC(B3LYP) 6-31G SP
```

MOLECULAR DYNAMICS (GROMACS)

- **Protein-Ligand Stability & Energetics:** Performed a 10 ns full MD production run of the target protein bound to the *jz4* ligand.
 - *Workflow:* System topology generation (**AMBER99SB-ILDN** force field, **TIP3P** water model), solvation in a dodecahedron box, charge neutralization (Na^+/Cl^-), Energy Minimization (steepest descent), and equilibration (NVT/NPT ensembles).
 - *Analysis:* Generated Backbone RMSD, RMSF, and Radius of Gyration ($R_g \approx 1.69$ nm) stability plots. Extracted short-range interaction energies (Lennard-Jones ≈ -98 kJ/mol vs. Coulombic ≈ -22 kJ/mol) revealing a hydrophobic-driven binding mechanism.



(a) Protein-Ligand Binding Pocket



(b) Backbone RMSD



(c) Residual Fluctuations (RMSF)

Figure 3: Structural and Dynamic Analysis of the Complex. (a) High-resolution render demonstrating the *jz4* ligand perfectly accommodated within the binding pocket. (b) RMSD trajectory confirming global system equilibration after 2 ns. (c) RMSF per residue highlighting the structural rigidity of the backbone and the local stabilization provided by the ligand.

Real-Time Stochastic Modeling

- **Animated Random Walks:** Developed a Python script to simulate and animate stochastic particle movement.
- **Implementation and Results** The simulation was implemented using a Python-based algorithm employing the NumPy library for efficient vectorization and Matplotlib for advanced data visualization. To observe the transition from individual steps to a diffusive "cloud" behavior, a high-iteration simulation was performed

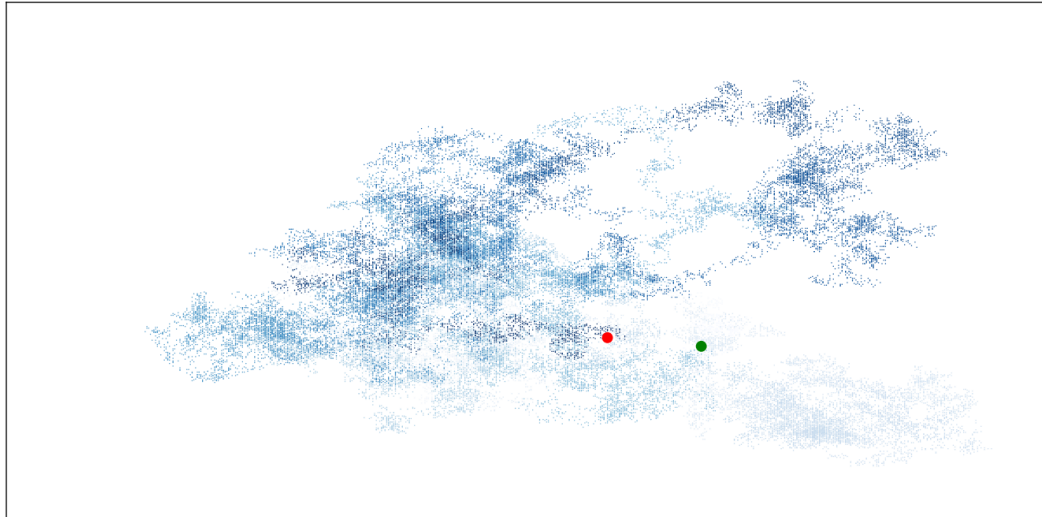


Figure 4: **2D Stochastic Process Visualization.** A Random Walk simulation consisting of 50,000 steps. The temporal evolution of the particle's trajectory is mapped through a cyan color gradient, where lighter shades represent earlier stages and darker shades indicate later stages. The starting position is marked with a green dot, and the final position is denoted by a red dot, illustrating the overall displacement generated by the pseudo-random number generator.

Listing 3: Class method for custom step calculation in RandomWalk

```
def get_step(self):
    """Calculate direction and distance for each stochastic iteration"""
    direction = choice([1, -1])
    distance = choice([0, 1, 2, 3, 4])
    step = direction * distance
    return step
```

Listing 4: Real-time animation implementation using Matplotlib

```
def update(frame):
    ax.clear()
    ax.scatter(rw.x_values[:frame], rw.y_values[:frame], s=10)
    ax.set_title(f"Simulation Step: {frame}")

# Creating the animation object
ani = FuncAnimation(fig, update, frames=len(rw.x_values), interval=0.1)
```

- *Technical Highlight:* Utilized `matplotlib.FuncAnimation` for real-time rendering, demonstrating efficient handling of iterative loops and data updating.

GEOSPATIAL DATA ANALYSIS: GLOBAL SEISMIC ACTIVITY

Data Context Mapping global earthquakes instantly outlines major tectonic boundaries, such as the Pacific Ring of Fire. The dataset utilized for this visualization was sourced from the US Geological Survey (USGS) live earthquake feed. By encoding earthquake magnitude into both color and marker size, this visualization transforms raw geospatial data into a clear, multidimensional analysis.

Global Earthquakes

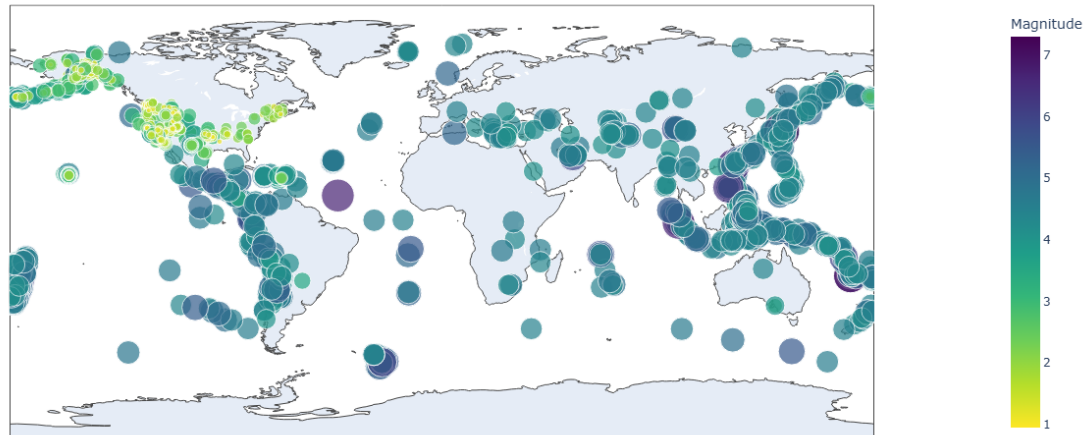


Figure 5: **Global Earthquake Distribution.** A geospatial scatter plot mapping seismic events worldwide. Spatial coordinates (latitude and longitude) determine positioning, while the earthquake's magnitude is double-encoded into both marker size and a sequential color gradient (ranging from yellow for minor events to dark purple for high-magnitude earthquakes ≥ 7). The data clustering distinctly outlines major tectonic fault lines.

Implementation Details The visualization was developed by parsing structured geospatial data (typically JSON or CSV formats provided by geological agencies like the USGS) containing the coordinates and metadata of seismic events. The implementation highlights several key data science skills:

- **Coordinate Projection:** Mapping spherical coordinates (latitude/longitude) onto a 2D base map.
- **Multidimensional Encoding:** Utilizing a customized continuous colormap and dynamic marker sizing to represent the continuous numerical variable of magnitude.
- **Data Ink Ratio:** Maintaining a clean aesthetic by removing unnecessary map details to allow the data points to be the focal point of the visual.

Listing 5: Parsing GeoJSON data and generating interactive geospatial visualizations

```
# Extracting magnitudes and coordinates from nested JSON structure
for eq_dict in all_eq_dicts:
    mags.append(eq_dict['properties']['mag'])
    lons.append(eq_dict['geometry']['coordinates'][0])
    lats.append(eq_dict['geometry']['coordinates'][1])
    hover_texts.append(eq_dict['properties']['title'])

# Defining interactive map with Plotly
data = [{
    'type': 'scattergeo',
    'lon': lons, 'lat': lats,
    'marker': {
        'size': [5*mag for mag in mags], # Dynamic scaling
        'color': mags,
        'colorscale': 'Viridis',
        'colorbar': {'title': 'Magnitude'},
    }
}]
```

TIME SERIES ANALYSIS: ANNUAL TEMPERATURE VARIATIONS

Data Context Analyzing meteorological patterns requires robust time series visualization to identify both macro-seasonal trends and micro-daily volatility. By plotting the daily maximum and minimum temperatures over a full year, the visualization captures overarching climatic shifts. The shaded area between the curves highlights the daily thermal amplitude, providing immediate insight into temperature variance.

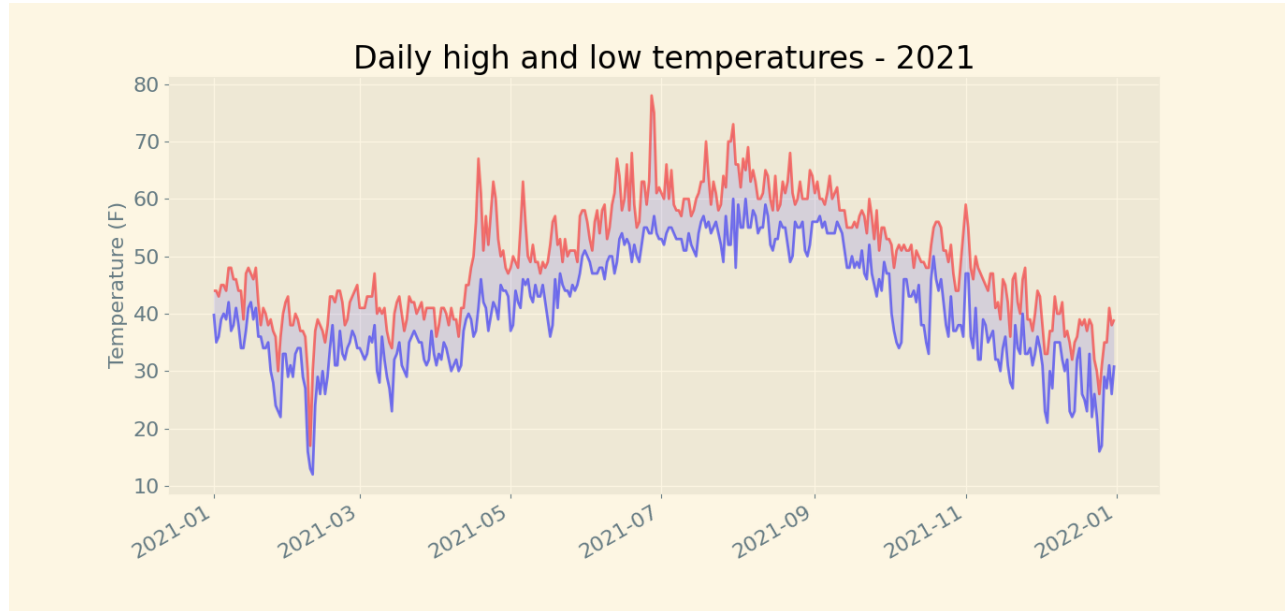


Figure 6: **Daily High and Low Temperatures (2021)**. A chronological plot displaying annual thermal fluctuations. The upper **red line** tracks daily highs, while the lower **blue line** tracks daily lows. The shaded region encodes the daily temperature range. The formatted temporal x-axis clearly demonstrates the progression of seasonal warming and cooling trends.

Implementation This visualization was developed by parsing sequential weather datasets, demonstrating proficiency in handling `datetime` structures and temporal data:

- **Temporal Parsing:** Converting string-based dates into standard chronological objects to ensure accurate scaling along the x-axis.
- **Range Visualization:** Employing dual-line plotting alongside the `fill_between` function to emphasize the magnitude of the daily temperature gap.
- **Formatting Aesthetics:** Applying a stylized grid background and rotating the chronological x-tick labels for optimal readability and a professional reporting aesthetic.

Listing 6: Data parsing and variance visualization from CSV weather logs

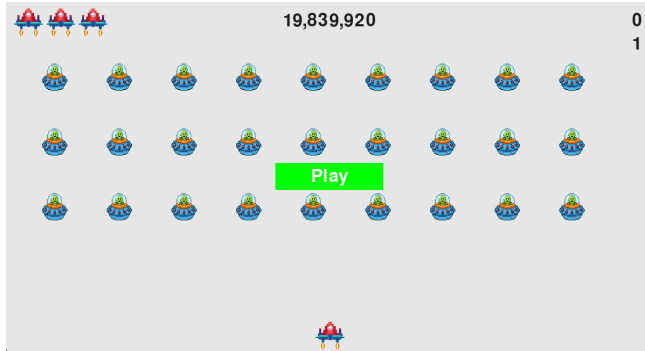
```
# Extracting dates, high and low temperatures with datetime conversion
for row in reader:
    current_date = datetime.strptime(row[2], '%Y-%m-%d')
    high, low = int(row[4]), int(row[5])
    dates.append(current_date)
    highs.append(high)
    lows.append(low)

# Plotting with shaded variance region
ax.plot(dates, highs, c='red', alpha=0.5)
ax.plot(dates, lows, c='blue', alpha=0.5)
ax.fill_between(dates, highs, lows, facecolor='blue', alpha=0.1)
```

GAME DEVELOPMENT: "ALIEN INVASION" (PYTHON/PYGAME)

Object-Oriented Programming (Game Dev)

- **Alien Invasion Project:** Developed a fully functional "Space Invaders" style game. Implemented complex class hierarchies, event handling for user input, and collision detection algorithms.



(a) Level Start: Full Fleet



(b) Active Gameplay: Combat Logic

Figure 7: **Alien Invasion State Progression.** (a) Initial game state showing the full alien fleet, player ship, and reset score, validating class initialization. (b) Combat phase demonstrating core mechanics: projectile physics, dynamic alien movement, collision detection, and real-time score updates managed by the core game loop.

Listing 7: Object-Oriented game logic and collision detection

```
def _check_bullet_alien_collisions(self):
    """Respond to bullet-alien collisions using Sprite Group methods"""
    collisions = pygame.sprite.groupcollide(
        self.bullets, self.aliens, True, True)

    if collisions:
        for aliens in collisions.values():
            self.stats.score += self.settings.alien_points * len(aliens)
            self.sb.prep_score()
            self.sb.check_high_score()

    if not self.aliens:
        self.bullets.empty()
        self._create_fleet()
        self.settings.increase_speed()
```


DATA ENGINEERING & API INTEGRATION: GITHUB REPOSITORY ANALYSIS

Project Overview Extracting and processing live data from web services is a fundamental skill in modern data engineering. In this project, the GitHub REST API was utilized to query, retrieve, and analyze the most popular open-source Python repositories based on community engagement (star counts).

Most-Starred Python Projects on GitHub

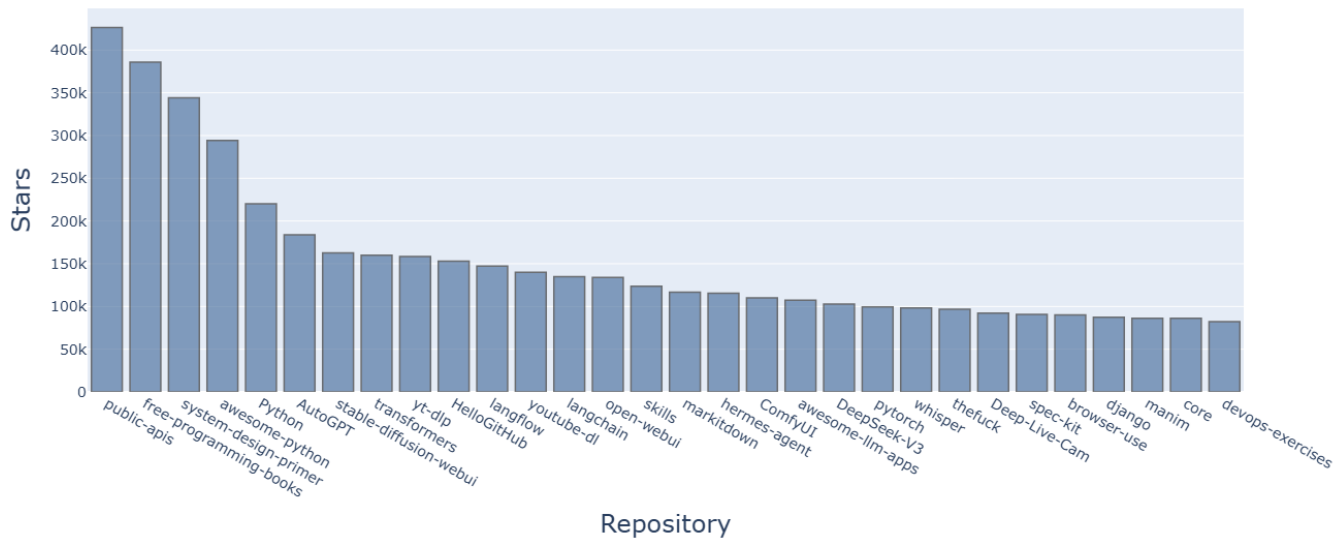


Figure 8: **Most-Starred Python Projects on GitHub.** A bar chart visualizing the top Python repositories ranked by their total number of stars. The data was dynamically fetched via the GitHub API, parsed from nested JSON responses, and plotted to highlight current trends in the open-source Python ecosystem.

Technical Implementation The development of this visualization required building a streamlined ETL (Extract, Transform, Load) pipeline:

- **API Interaction:** Formulating GET requests to the GitHub API, effectively handling response status codes, and managing data payloads.
- **Data Parsing (JSON):** Extracting specific attributes (repository names and star counts) from complex, deeply nested JSON dictionary structures and transforming them into plottable arrays.
- **Visualization Aesthetics:** Designing a readable bar chart by implementing rotated x-tick labels for long string names and scaling the y-axis dynamically to handle hundreds of thousands of units cleanly.

Listing 8: REST API integration and data retrieval from GitHub

```
# API call configuration for Python repositories
url = 'https://api.github.com/search/repositories?q=language:python&sort=stars'
headers = {'Accept': 'application/vnd.github.v3+json'}
r = requests.get(url, headers=headers)

# Processing JSON response and extracting metadata
response_dict = r.json()
repo_dicts = response_dict['items']
for repo_dict in repo_dicts:
    stars.append(repo_dict['stargazers_count'])
    owner = repo_dict['owner']['login']
    description = repo_dict['description']
```

Code samples and full repositories available at: github.com/diegrz